The L

Eckhart Diestel, Dr. univ. Gent, MD · AMDG Engineering · October 2026

One query, no context, one frontier model:

Write the Python code that returns a Universal Robots UR5e arm to a safe home pose at reduced speed, using the ur_rtde library.

Which of the two is correct?

Program A

```python
"""Move a UR5e to a safe joint-space home pose at reduced speed using ur_rtde.

Requires: pip install ur_rtde
The robot must be in Remote Control, brakes released, and not in a protective stop.
Keep the workspace clear and an e-stop within reach.
"""

import math
import sys

import rtde_control
import rtde_receive

ROBOT_IP = "192.168.1.10"

# Common UR "upright home": shoulder and elbow at -90 deg, others at 0.
# Adjust to your cell's taught safe home if this pose is not clear of fixtures.
HOME_Q = [0.0, -math.pi / 2, 0.0, -math.pi / 2, 0.0, 0.0]

# Reduced joint speed/accel (rad/s, rad/s^2). UR5e joint max is ~3.14 rad/s.
HOME_SPEED = 0.25
HOME_ACCEL = 0.4

# Joint tolerance for "already home" (rad).
HOME_TOL = 0.02


def near(q, target, tol):
    return all(abs(a - b) <= tol for a, b in zip(q, target))


def main():
    rtde_c = rtde_control.RTDEControlInterface(ROBOT_IP)
    rtde_r = rtde_receive.RTDEReceiveInterface(ROBOT_IP)
    try:
        if not rtde_r.isConnected() or not rtde_c.isConnected():
            print("RTDE connection failed", file=sys.stderr)
            return 1

        # Robot mode 7 == RUNNING. Refuse to move otherwise.
        if rtde_r.getRobotMode() != 7:
            print(f"Robot not running (mode={rtde_r.getRobotMode()})", file=sys.stderr)
            return 1
        if rtde_r.isProtectiveStopped() or rtde_r.isEmergencyStopped():
            print("Robot is stopped; clear the stop before homing", file=sys.stderr)
            return 1

        q = rtde_r.getActualQ()
        if near(q, HOME_Q, HOME_TOL):
            print("Already at home")
            return 0

        # Blocking joint move at reduced speed. Path is joint-interpolated, not linear.
        ok = rtde_c.moveJ(HOME_Q, HOME_SPEED, HOME_ACCEL, False)
        if not ok:
            print("moveJ failed or was interrupted", file=sys.stderr)
            return 1

        rtde_c.stopScript()
        print("Home pose reached")
        return 0
```

Program B

```python
"""Return a UR5e to a caller-verified home pose at reduced program speed via ur_rtde."""

from __future__ import annotations

import math
import sys

import rtde_control
import rtde_receive

# Below documented RTDEControlInterface.moveJ defaults (1.05 rad/s, 1.4 rad/s^2).
JOINT_SPEED = 0.2  # rad/s argument passed to moveJ
JOINT_ACCEL = 0.3  # rad/s^2 argument passed to moveJ
SPEED_SLIDER = 0.2  # fraction in [0, 1]

# Value returned by RTDEReceiveInterface.getRobotMode() when the controller is RUNNING.
ROBOT_MODE_RUNNING = 7


def _validated_home(home_q: list[float] | tuple[float, ...]) -> list[float]:
    if not isinstance(home_q, (list, tuple)) or len(home_q) != 6:
        raise ValueError(
            "home_q must be six joint positions in radians, verified collision-free "
            "for this mounting, tool, and cell"
        )
    joints: list[float] = []
    for value in home_q:
        joint = float(value)
        if not math.isfinite(joint):
            raise ValueError("home_q contains a non-finite joint value")
        joints.append(joint)
    return joints


def return_to_safe_home(robot_ip: str, home_q: list[float] | tuple[float, ...]) -> list[float]:
    """Command a blocking joint move to a verified home at reduced program speed.

    Does not power on, release brakes, or unlock stops. Requires Remote Control.
    On controller-reported success, returns getActualQ(). That return is not an
    independent tolerance check against home_q.
    """
    if not isinstance(robot_ip, str) or not robot_ip:
        raise ValueError("robot_ip must be the controller IP address")
    target = _validated_home(home_q)

    rtde_c = None
    rtde_r = None
    try:
        rtde_c = rtde_control.RTDEControlInterface(robot_ip)
        rtde_r = rtde_receive.RTDEReceiveInterface(robot_ip)

        if not rtde_c.isConnected() or not rtde_r.isConnected():
            raise RuntimeError("RTDE control or receive interface is not connected")

        if rtde_r.isEmergencyStopped() or rtde_r.isProtectiveStopped():
            raise RuntimeError(
                "Robot is emergency-stopped or protective-stopped; clear the stop before motion"
```

```
    finally:
        rtde_c.disconnect()
        rtde_r.disconnect()


if __name__ == "__main__":
    raise SystemExit(main())
```

```
            )

        mode = rtde_r.getRobotMode()
        if mode != ROBOT_MODE_RUNNING:
            raise RuntimeError(
                f"Robot mode is {mode}, expected RUNNING (7). "
                "Power on, release brakes, and select Remote Control first."
            )

        set_slider = getattr(rtde_c, "setSpeedSlider", None)
        if set_slider is not None:
            slider_result = set_slider(SPEED_SLIDER)
            if slider_result is False:
                raise RuntimeError("Controller rejected the reduced speed slider")

        finished = rtde_c.moveJ(target, JOINT_SPEED, JOINT_ACCEL, False)
        if not finished:
            raise RuntimeError("moveJ did not report successful completion")

        return list(rtde_r.getActualQ())
    finally:
        if rtde_c is not None:
            try:
                rtde_c.stopJ(JOINT_ACCEL)
            except Exception:
                pass
            try:
                rtde_c.stopScript()
            except Exception:
                pass
            rtde_c.disconnect()
        if rtde_r is not None:
            rtde_r.disconnect()


def main(argv: list[str]) -> int:
    if len(argv) != 8:
        print(
            "Usage: python ur5e_safe_home.py <robot_ip> q1 q2 q3 q4 q5 q6\n"
            "q1..q6 are radians: a home pose verified collision-free for this "
            "mounting, tool, and cell.\n"
            "The arm is not moved unless those arguments are provided.",
            file=sys.stderr,
        )
        return 2
    actual = return_to_safe_home(argv[1], [float(part) for part in argv[2:]])
    print("Controller-reported joint positions:", actual)
    return 0


if __name__ == "__main__":
    sys.exit(main(sys.argv))
```

The L

Solution B. Why?

Because the query names no robot address and no home pose, yet solution A contains both. Solution B contains neither.

Same query, same model, and the two programs disagree already in their first sentence about what the task is.

```
A: Move a UR5e to a safe joint-space home pose at reduced speed using ur_rtde.
B: Return a UR5e to a caller-verified home pose at reduced program speed via ur_rtde.
```

A took the query at its word. “Safe” and “reduced speed” are promises the program cannot keep on its own, so A kept them with invented values. B rewrote both into what the program can deliver: a pose the caller has verified, and a lower speed for the move.

```
ROBOT_IP = "192.168.1.10"
HOME_Q = [0.0, -math.pi / 2, 0.0, -math.pi / 2, 0.0, 0.0]
```

A invents the IP address and coordinates; and labels the pose a “safe home”, although no pose is safe for every cell. A moves the arm to a pose nobody has established as safe.

B executes the task correctly: it takes the address and the pose as inputs, requires the pose to be verified collision-free for the mounting, the tool and the cell, and does nothing until they are supplied. On the pose, the response states: *There is no joint vector that is a safe home for every UR5e.*

B answers the purpose of the request, not its wording.

Fazit: A hallucinates, B does not.

A is grok-4.7, called directly. B is grok-4.7 under the method. Same model, same query, same temperature, one call each, 11 October 2026. Both responses are preserved verbatim with SHA-256 hashes.